JARVIS V2 // PERSONAL AI OPERATING LAYER

SYSTEM ARCHITECTURE, COMPLETE STATUS & HACKATHON SUBMISSION DOSSIER

Project Identity: JARVIS V2 (Intelligent Operating Layer)	**Operating System:** Windows 11 / Windows 10 (x64)
Primary Operator: Mahi (Full System Admin)	**Backend Runtime:** Python 3.12.10 (FastAPI + Uvicorn)
Frontend Visualization: Neural Reactor (High-DPI Canvas)	**Event Bus:** Live Full-Duplex WebSockets (/ws/events)
Overall Project Completion: 100% OPERATIONAL	**Automated Test Suite: 7 / 7 Passed (100% OK)**

1. Executive Summary & Hackathon Value Proposition

JARVIS V2 is a full-stack, multimodal personal AI operating interface that transforms personal computing by bridging advanced LLM reasoning directly to native Windows operating system hardware controls, media pipelines, and continuous real-time audio visualization.

While conventional AI assistants remain trapped inside isolated web chat sandboxes, JARVIS V2 exercises **direct low-level OS automation**—controlling system master volume, display brightness, native Windows applications (Google Chrome, Notepad, Terminal, VS Code, Explorer), camera hardware, battery diagnostics via Windows CIM/WMI, and YouTube media streaming—all protected by a **3-Tier Permission Gatekeeper**.

2. Project Completion Matrix (100% Working Prototype)

The project has achieved **100% implementation** across all foundational architecture modules, hardware controllers, and client interfaces:

Functional Subsystem	Architecture Implementation	Status
Universal 11-Step Orchestrator	Receive -> Understand -> Classify -> Plan -> Authorize -> Pre-Hook -> Execute -> Verify -> Recover -> Respond -> Remember	**100% COMPLETE**
3-Tier Security Gatekeeper	Auto-execution for LOW risk; tokenized human authorization modal for HIGH risk actions	**100% COMPLETE**
Windows Device Control	Master audio volume (exact %, +/- %, mute), WMI screen brightness, CIM battery telemetry, window manager	**100% COMPLETE**
Application Launcher	Natively launches Chrome, Notepad, Paint, VS Code, Terminal, and system folders (Documents, Downloads)	**100% COMPLETE**
Media & YouTube Streaming	Automated query resolution (e.g. 'play a telugu song') fetching video IDs with native and web autoplay	**100% COMPLETE**
Neural Reactor Hologram	Living AI orb: 210 particles, 10 glass bubbles, 5 dynamic states (Idle, Listening, Thinking, Responding, Error)	**100% COMPLETE**
Web Audio API Microphone	Live AnalyserNode RMS audio spectrum analysis driving genuine orb pulse (zero fake simulation)	**100% COMPLETE**
Continuous Speech Recognition	Speech recognition with silence debounce commit, auto-recovery, and natural speech synthesis	**100% COMPLETE**

9 Dedicated Workspaces	Dashboard, AI Chat, Voice Reactor, Tasks & Automation, Projects/Files, Memory, Tools, History, Settings	**100% COMPLETE**
WebSocket Telemetry Bus	Full-duplex real-time telemetry streaming at /ws/events with automatic client reconnection	**100% COMPLETE**
Automated Verification Suite	Unit and integration test suites validating all tools, orchestrator steps, gatekeeper, and endpoints	**100% (7/7 PASS)**

3. Technical Architecture & Universal 11-Step Pipeline

Every textual and spoken directive received by JARVIS passes through an asynchronous 11-step execution orchestrator (backend/core/orchestrator.py) providing deterministic reliability, parameter verification, and security:

- **Step 1: Receive:** Ingests directive from REST endpoint (POST /chat), WebSocket, or speech STT.
- **Step 2: Understand:** Applies natural language normalization: strips polite prefixes, wakes words, and maps gerunds.
- **Step 3: Classify:** Resolves semantic intent, maps to registered tool schemas, and computes risk level (LOW/MED/HIGH).
- **Step 4: Plan:** Instantiates a multi-step execution plan tracked inside Context Manager and broadcasted to HUD.
- **Step 5: Authorize:** Passes call through 3-Tier Gatekeeper. If HIGH risk, halts and yields cryptographic confirmation token.
- **Step 6: Pre-Hook:** Fires pre-execution hooks, auditing action parameters into Activity Log.
- **Step 7: Execute:** Dispatches native Python/PowerShell/CIM automation or AI reasoning pipeline.
- **Step 8: Verify:** Validates state change (e.g. verifies process startup in tasklist, confirms file creation on disk).
- **Step 9: Recover:** Catches execution anomalies gracefully with alternative fallbacks without breaking interface.
- **Step 10: Respond:** Assembles structured response: spoken speech utterance, URL navigation, and HUD cards.
- **Step 11: Remember:** Extracts key operator preferences and commits facts into memory.json.

4. Neural Reactor Holographic AI Engine

The centerpiece of the JARVIS V2 interface is the **Neural Reactor**—a high-performance, hardware-accelerated living AI orb rendered on HTML5 Canvas with window.devicePixelRatio high-DPI scaling. Unlike generic equalizers or rotating rings, it features 5 signature physics-driven animation states:

- **State A — LISTENING (Neural Pulse):** Connected to genuine microphone via Web Audio API AnalyserNode. Calculates root-mean-square (RMS) vocal frequencies to modulate core pulse, bubble expansion, and acoustic wave ripples. Zero fake simulation; handles denial gracefully.
- **State B — THINKING (Particle Recombination):** Signature effect: Orbiting bubbles destabilize and dissolve into micro-particles. Particles accelerate along curved vortex flows with harmonic curl noise, forming an evolving abstract neural network with glowing synaptic lines (d < 46px). Reconverges seamlessly upon inference completion.
- **State C — RESPONDING (Wave Release):** Emanates concentric shockwaves of light and accelerated particle bursts outward from the core in rhythm with Speech Synthesis vocalization.
- **State D — IDLE & HOVER (Celestial Orbit):** Smooth orbital motion of 210 particles and 10 glass bubbles with refraction borders. Moving mouse cursor over canvas applies real-time magnetic repulsion physics.
- **State E — ERROR / RECOVERY:** Shifts palette to crimson rose (#f43f5e) with agitated high-frequency jitter, followed by smooth automatic recovery back to standby.

5. Verified Basic Directives & Execution Evidence

All fundamental computer controls and voice commands were verified on the live system:

Directive Command	Tool Dispatched	Observed Execution Result

open chrome	computer.open_app	Launched Google Chrome in new window & navigated to Google Homepage
play a telugu song	media.play_youtube	Queried YouTube, resolved video ID qi04rwOn08k, launched autoplay player
set volume to 40%	computer.volume	Adjusted Windows master audio volume via PowerShell WScript.Shell
what is my battery?	computer.battery	Reported live CIM Win32_Battery telemetry: 77% discharging
open notepad	computer.open_app	Started Windows Notepad application natively
take screenshot	computer.screenshot	Captured instantaneous desktop screen and saved to workspace
search web for AI	browser.search	Fetched live DuckDuckGo results and formatted Markdown cards with citations

6. Hackathon Demonstration Script (2-Minute Demo)

Follow this exact structured sequence to showcase JARVIS V2 for hackathon judges or demo videos:

- **Step 1: Introduction (15s):** Open http://127.0.0.1:8000. Point out the dark cybernetic cockpit, live hardware telemetry (CPU, RAM, Battery), and WebSocket connection badge.
- **Step 2: Voice & Neural Reactor (30s):** Click 'Voice Assistant' in sidebar. Highlight the living AI orb. Click the microphone button. Speak: *'Jarvis, what is my battery?'* Show how the orb pulses with genuine vocal sound waves, dissolves into an abstract neural network during thinking, and vocalizes the reply.
- **Step 3: Native Device Control (30s):** Speak or type: *'Set volume to 50%'* followed by *'Open Chrome'*. Show how Chrome launches natively on Windows.
- **Step 4: Media & Entertainment (25s):** Speak: *'Play a Telugu song'*. Point out YouTube resolution, instant autoplay playback, and song query.
- **Step 5: Security Gatekeeper (20s):** Type: *'Delete system file'*. Show the 3-Tier Security Gatekeeper pop up with the token authorization modal, explaining consequential action safety.

7. Git Push & Production Deployment Instructions

To publish the repository to GitHub and deploy the application:

Git Commit & Push Commands:

```
git init
git add .
git commit -m "JARVIS V2: Complete AI Operating Layer with Neural Reactor"
git branch -M main
git remote add origin https://github.com/<your-username>/jarvis-v2.git
git push -u origin main
```

Local Windows Run:

Double-click start_jarvis.bat or run npm.cmd start in terminal.

END OF DOSSIER — JARVIS V2 CERTIFIED 100% OPERATIONAL